

Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization

A Summary of Huang & Belongie (2017), with Mathematical Historical Context

The idea

AdaIN rests on one claim. The style of an image is carried by the per-channel mean and variance of its deep features. If that is true, you do not need to optimize toward a style, and you do not need to train a network per style. You compute the style's channel statistics, write them onto the content features in closed form, and decode. One forward pass. Any style [8].

Like the Gatys method, this did not arrive from nowhere. It is the endpoint of two mathematical lines. One made stylization fast by amortizing it into a trained network. The other, the line of normalization layers, slowly discovered that a channel's mean and variance are a handle on style. AdaIN is where they meet. Walk each line first.

The first line: fast feed-forward stylization

Gatys made style transfer beautiful and slow [1]. Every image was a fresh optimization of the pixels, minutes long. The obvious fix was to pay that cost once, in advance, and amortize it. Johnson, Alahi, and Fei-Fei did exactly that [2]. They trained a feed-forward transformation network f_W so that a single pass produced the stylized image, minimizing, over a corpus of content images, the same perceptual and Gram losses Gatys used,

$$W^* = \arg \min_W \mathbb{E}_c \left[\alpha \mathcal{L}_{\text{content}}(f_W(c), c) + \beta \mathcal{L}_{\text{style}}(f_W(c), s) \right].$$

Ulyanov and colleagues reached the same idea in parallel with Texture Networks [3]. Both bought speed. Both paid for it with rigidity: the style s is baked into the weights W^* , so one network serves exactly one style. A new style means training again. AdaIN inherits the feed-forward decoder from this line and removes the rigidity.

The second line: normalization layers

The deeper line is normalization. Every normalization layer these methods use is the same operation with two open choices,

$$\text{Norm}(x) = \gamma \odot \frac{x - \mu(x)}{\sigma(x)} + \beta.$$

The choices are: over which axes you compute the statistics μ, σ , and where the affine scale γ and shift β come from. The whole history of AdaIN is a walk through those two choices.

Batch normalization came first, to stabilize training [4]. It computes each channel's statistics across the batch and the spatial positions,

$$\mu_c(x) = \frac{1}{NHW} \sum_{n,h,w} x_{nchw}, \quad \sigma_c(x) = \sqrt{\frac{1}{NHW} \sum_{n,h,w} (x_{nchw} - \mu_c(x))^2 + \epsilon},$$

with γ_c, β_c learned constants. The batch is averaged in, which is fine for recognition and wrong for style, because it mixes the statistics of different images.

Instance normalization dropped the batch axis [5]. It standardizes each sample on its own, one channel at a time, over spatial positions only,

$$\mu_{nc}(x) = \frac{1}{HW} \sum_{h,w} x_{nchw}, \quad \sigma_{nc}(x) = \sqrt{\frac{1}{HW} \sum_{h,w} (x_{nchw} - \mu_{nc}(x))^2 + \epsilon}.$$

Swapping batch normalization for this sharply improved feed-forward stylization. Huang and Belongie later gave the reason: instance normalization performs a kind of style normalization. Standardizing a channel erases the per-sample feature statistics, which is to say it erases the input’s own style, leaving the learned γ, β to write a single style back in [8].

Conditional instance normalization made that last point explicit [6]. Keep the same normalization, but learn a separate affine pair for each style s ,

$$\text{CIN}(x; s) = \gamma^s \odot \frac{x - \mu(x)}{\sigma(x)} + \beta^s.$$

One network now held many styles, and the only thing that changed between them was the vector of affine parameters, $2C$ numbers per layer. The conclusion was hard to miss. Style is encoded, almost entirely, in (γ^s, β^s) .

Layer	Statistics computed over	Scale γ , shift β	Styles
BN [4]	batch N , spatial H, W	learned constants	none
IN [5]	spatial H, W , per instance	learned constants	one
CIN [6]	spatial H, W , per instance	learned per style s	a fixed set
AdaIN [8]	spatial H, W , per instance	from the style: $\sigma(y), \mu(y)$	any

The theory: statistics are style

Why should two numbers per channel suffice? The texture-synthesis tradition had long held that appearance is a matter of feature statistics. The modern statement came from Li and colleagues, who proved that matching Gram matrices, as Gatys does, is equivalent to minimizing the maximum mean discrepancy between two feature distributions under a polynomial kernel [7]. Style matching is distribution matching. They also showed that matching simpler statistics, the channel-wise mean and variance, serves as a valid style objective in its own right. That result is the licence for AdaIN. If mean and variance are enough to name a style, then setting them is enough to transfer one.

The confluence: adaptive instance normalization

Now the lines meet. Take the feed-forward decoder from the first line. Take the normalization operation from the second, at the end of its progression, and make the last choice adaptive. Instead of learning γ, β , read them from the style image’s own feature statistics: set $\gamma = \sigma(y)$ and $\beta = \mu(y)$. For content features x and style features y ,

$$\text{AdaIN}(x, y) = \sigma(y) \frac{x - \mu(x)}{\sigma(x)} + \mu(y)$$

The layer has no learned parameters. The style enters as an input, not as a weight. This is the bottom row of the table, and it is the whole method.

The architecture wraps it in a sandwich. A fixed VGG encoder f , up to `relu4_1`, reads both images. Align them, then decode,

$$t = \text{AdaIN}(f(c), f(s)), \quad T(c, s) = g(t).$$

The decoder g mirrors the encoder, with no normalization layers of its own, and is trained once. Its objective is consistent with everything above. The content term keeps the output near the aligned features t ; the style term matches the same statistics AdaIN aligns and Li’s theory justified, the mean and standard deviation across several VGG layers ϕ_i ,

$$\mathcal{L}_c = \|f(g(t)) - t\|_2, \quad \mathcal{L}_s = \sum_i \|\mu(\phi_i(g(t))) - \mu(\phi_i(s))\|_2 + \sum_i \|\sigma(\phi_i(g(t))) - \sigma(\phi_i(s))\|_2,$$

with total loss $\mathcal{L} = \mathcal{L}_c + \lambda \mathcal{L}_s$. Train the decoder on many content and style pairs, and afterward any new style transfers with no retraining.

What it showed

AdaIN is the first method that is both fast and open to any style. The speed comes from the feed-forward line [2, 3]. The freedom comes from making the affine parameters a function of the input rather than trained constants, which the normalization line had been circling for two years [4, 5, 6].

It also settled an old puzzle. Instance normalization worked because it strips style, and AdaIN works because it puts a chosen style back; the earlier trick was never a mystery once you saw the affine parameters as the style itself [8]. Because the style is now just a scale and a shift computed at test time, you get control for free: interpolate between the content features and the AdaIN output to trade content against style, or mix several AdaIN outputs to blend styles, all without retraining.

The honest limit is fidelity. AdaIN aligns only the per-channel mean and variance, the diagonal of the second-order statistics, and drops the cross-channel correlations that the Gram matrix keeps. Work that followed closed part of that gap by aligning the full feature covariance through whitening and coloring transforms [9], at more cost. AdaIN keeps the diagonal, and keeps the speed. For most styles, most of the time, the diagonal is enough.

References

- [1] L. A. Gatys, A. S. Ecker, and M. Bethge. A Neural Algorithm of Artistic Style. arXiv:1508.06576, 2015.
- [2] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual Losses for Real-Time Style Transfer and Super-Resolution. ECCV, 2016.
- [3] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky. Texture Networks: Feed-forward Synthesis of Textures and Stylized Images. ICML, 2016.
- [4] S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML, 2015.
- [5] D. Ulyanov, A. Vedaldi, and V. Lempitsky. Instance Normalization: The Missing Ingredient for Fast Stylization. arXiv:1607.08022, 2016.
- [6] V. Dumoulin, J. Shlens, and M. Kudlur. A Learned Representation for Artistic Style. ICLR, 2017.
- [7] Y. Li, N. Wang, J. Liu, and X. Hou. Demystifying Neural Style Transfer. IJCAI, 2017.
- [8] X. Huang and S. Belongie. Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization. ICCV, 2017.
- [9] Y. Li, C. Fang, J. Yang, Z. Wang, X. Lu, and M.-H. Yang. Universal Style Transfer via Feature Transforms. NeurIPS, 2017.